

Contents

1	Chapter 10 — Population-Based Training and Evolutionary Game Theory: Experiment Report	1
2	PART I — EVOLUTIONARY GAME THEORY AS A DIAGNOSTIC LENS	2
2.1	1. What this chapter is about	2
2.2	2. Experiment 1 — replicator dynamics on four matrix games	2
2.3	3. Experiment 2 — the spinning-top: transitive vs cyclic structure	4
3	PART II — THE PBT LEAGUE	6
3.1	4. What the league is and how it is graded	6
3.2	5. Experiment 3 — league training dynamics	7
3.3	6. Experiment 4 — EGTA, meta-Nash, and diversity	8
3.4	7. Experiment 5 — league vs PSRO vs self-play vs CFR-Nash	10
3.5	8. Prediction <-> reality reconciliation	12
3.6	9. Trustworthiness and sample adequacy	13
3.7	10. Limitations (ranked by how much they affect the conclusions)	14
3.8	11. Conclusions and research directions	14
3.9	12. Reproduction	15

1 Chapter 10 — Population-Based Training and Evolutionary Game Theory: Experiment Report

Testbeds: four symmetric matrix games for the evolutionary-dynamics half (Prisoner’s Dilemma, Hawk-Dove, Rock-Paper-Scissors, Stag Hunt); a synthetic pure-skill ladder; the PSRO best-response meta-game on Leduc Hold’em; and a small AlphaStar-style **PBT league** of neural PPO agents playing Leduc Hold’em. The Leduc engine, the exact best-response oracle, and NashConv/exploitability come from the Chapter 07 stack (reused wholesale); PSRO and the meta-Nash solver come from Chapter 09. Neural agents are trained with PyTorch and then extracted to *tabular* policies so the Chapter 07 **exact** best response can grade them.

PhD connection: the population-level layer of the thesis. Three hooks: **main exploiters as automated opponent modelers** (Contribution #1, the population lift of Chapter 07’s Bayesian read); the **AlphaStar league as a population safe-exploitation mechanism whose guarantee is missing** (Contribution #2); and **EGTA / meta-Nash as the population evaluation methodology** (Contribution #3).

Scope of results: every number in this report is **measured from a real run** and read from the artifacts under `implementation/step10/implementation/results/{smoke,scale}_results.json` and `implementation/step10/exploration/figures/*.json`. Evolutionary results are bracketed by *analytic* references (ESS / Nash of each matrix game); league results are bracketed by Chapter 07’s *exact* exploitability. Three Phase-4 predictions were **contradicted** by the scale run; per WORKFLOW §0.1 they are kept as stated and reconciled with what actually happened (§8).

How to read this report. Both parts follow the same arc: **what we test** -> **how** -> **results** -> **conclusion**. **Part I (§1-§3)** uses evolutionary game theory as a *diagnostic lens*: replicator dynamics on solvable matrix games, then the transitive/cyclic “spinning-top” decomposition. **Part II (§4-§7)** builds and evaluates the **PBT league**: training dynamics, EGTA/meta-Nash, diversity + Elo, and a head-to-head against PSRO / self-play / CFR-Nash. §8 reconciles the three contradicted predictions; §9-§11 cover trust, limitations, and directions; §12 lists reproduction commands.

2 PART I — EVOLUTIONARY GAME THEORY AS A DIAGNOSTIC LENS

2.1 1. What this chapter is about

Chapters 2-9 reasoned about equilibria of a *fixed* game. Chapter 10 moves up a level to **populations of strategies that change over time** and asks two questions a single-agent view cannot: *which strategies grow?* (replicator dynamics) and *does this game even have a “best” strategy, or only a wheel of counters?* (the transitive/cyclic decomposition). These two tools are the diagnostic that Part II’s population-based training needs — because whether a self-training population converges or cycles is decided by the game’s structure, not the learning rate. All Part I evolutionary numbers are deterministic (fixed seeds, exact dynamics), so “did it converge?” is unambiguous.

2.2 2. Experiment 1 — replicator dynamics on four matrix games

What we test. Do the replicator dynamics reproduce each game’s *analytic* evolutionary outcome — convergence to a dominant strategy, to an interior ESS, to a basin-dependent pure ESS, or a non-convergent orbit? (Raw step validation, L530-533.)

How. Discrete-Euler replicator integration from a seeded interior start; “converged” means the last iterate stopped moving; “orbit radius” is the distance the trajectory holds from the uniform point. Analytic ESS/Nash is the reference. Data: `results/smoke_results.json` (replicator block), phase portraits in `exploration/figures/replicator_playground.json`.

Game	Analytic reference	Start x_0	Final x	Converged?	Orbit radius
Prisoner's Dilemma	Defect dominates; unique ESS (0, 1)	[0.666, 0.334]	[0.0, 1.0]	yes	0.707
Hawk-Dove	interior ESS $p(\text{Hawk}) =$ $V/C = 0.5$	[0.548, 0.452]	[0.5, 0.5]	yes	0.0
Rock- Paper- Scissors	Nash = uniform; no ESS (centre)	[0.347, 0.385, 0.268][0.299, 0.288, 0.413]		no	0.095
Stag Hunt ($x_0 =$ [0.8, 0.2])	two pure ESS (basin- dependent)	[0.8, 0.2]	[1.0, 0.0]	yes	0.707
Stag Hunt ($x_0 =$ [0.2, 0.8])	two pure ESS (basin- dependent)	[0.2, 0.8]	[0.0, 1.0]	yes	0.707

Results. All four match theory exactly. Prisoner's Dilemma collapses to all-Defect (the dominant strategy). Hawk-Dove converges to the interior mixed ESS at exactly 0.5 (orbit radius 0.0 — it *reaches* the rest point). Rock-Paper-Scissors **does not converge**: it orbits the uniform centre (radius ≈ 0.095), because the interior fixed point is a centre, not an attractor. In continuous time the orbits are closed; the explicit Euler integration used here turns them into a slowly widening spiral, so the radius drifts upward slightly over the run. Stag Hunt lands on *different* pure ESS depending on the start — $[0.8, 0.2] \rightarrow$ all-Stag, $[0.2, 0.8] \rightarrow$ all-Hare — the two basins made concrete.

Conclusion. Replicator dynamics are the continuous idealization of population selection: every Nash equilibrium is one of their rest points, and every ESS an asymptotically stable one (Hofbauer & Sigmund, 2003; the converse does not hold in general, e.g. every pure strategy is a rest point). The one that *refuses to converge* — RPS — is the whole reason Part II exists: a cyclic game has no stable population, so a self-training population in it will spin rather than settle.

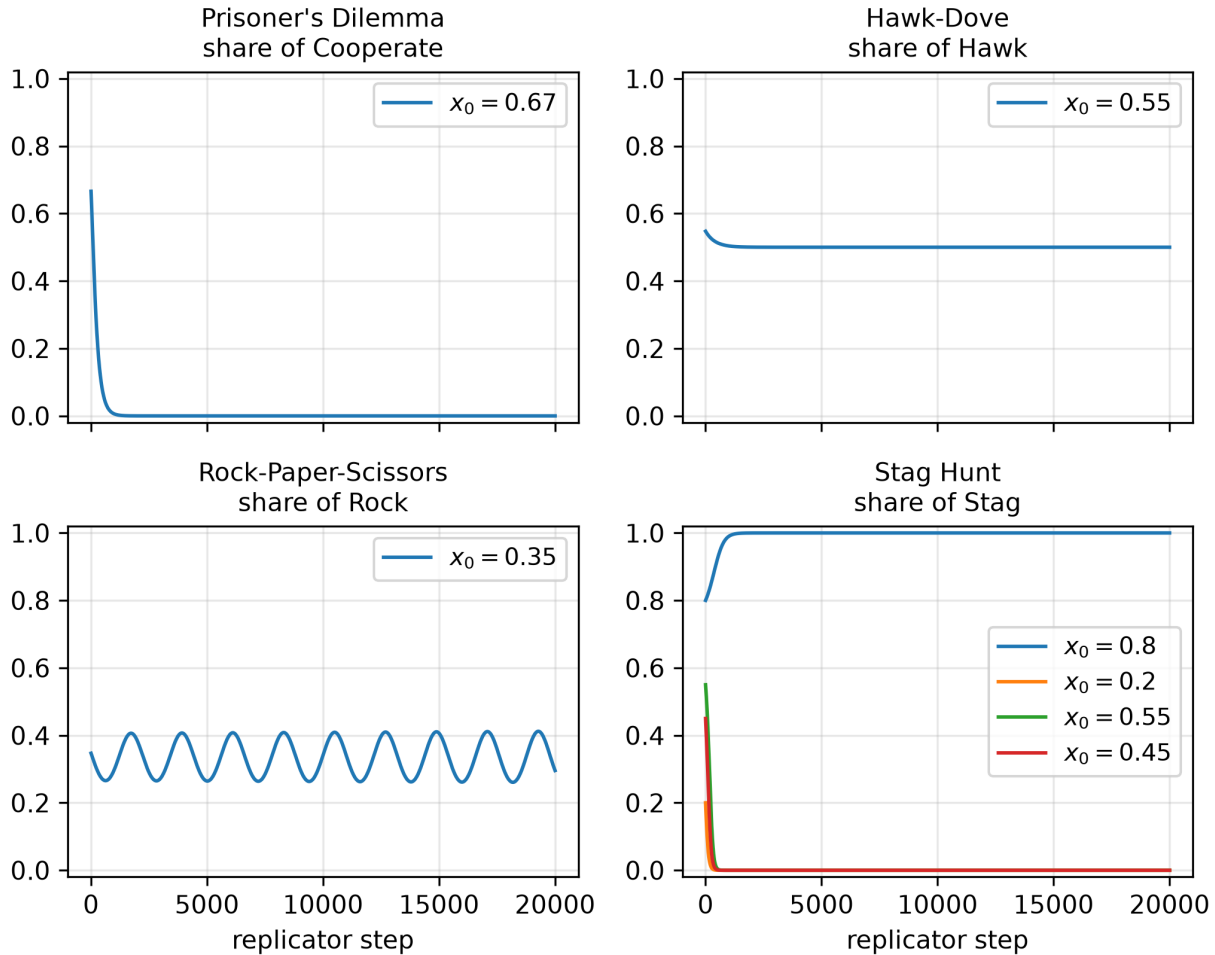


Figure 1: Replicator trajectories (share of the first strategy over time) on the four matrix games: Prisoner’s Dilemma collapses to all-Defect, Hawk-Dove converges to the interior 0.5 ESS, Rock-Paper-Scissors orbits the centre without converging, and Stag Hunt splits into two basins (all-Stag or all-Hare) depending on the start. The non-converging RPS orbit is the visceral case for the diversity machinery in Part II.

2.3 3. Experiment 2 — the spinning-top: transitive vs cyclic structure

What we test. Can we decompose a game’s payoff matrix into a **transitive** (skill-ladder) component and a **cyclic** (rock-paper-scissors) component, and does the decomposition correctly identify pure cases and diagnose real populations? (Raw step validation, L532, L534.)

How. The combinatorial-**Hodge** (ratings-difference) decomposition on the antisymmetrized payoff matrix, reporting a transitive ratio and a cyclic ratio (orthogonal, so they combine in quadrature to 1). We also report the original plan’s SVD rank-1 transitive ratio for RPS to docu-

ment its known failure. Data: `results/{smoke,scale}_results.json` (spinning_top block), `exploration/figures/game_landscape.json`.

Population	Transitive ratio (Hodge)	Cyclic ratio (Hodge)	Three-cycles	Note
Rock-Paper-Scissors	0.0	1.0	1	SVD rank-1 wrongly gives 0.707
Pure skill ladder	1.0	0.0	0	monotone strength
PSRO-Leduc meta-game (smoke, 8 rounds)	0.457	0.890	—	mostly cyclic
PSRO-Leduc meta-game (scale, 20 rounds)	0.411	0.912	27	mostly cyclic
League snapshot meta-game (smoke / scale)	0.981 / 0.937	—	—	mostly transitive

Results. The Hodge decomposition nails the pure cases: RPS is 0.0 transitive / 1.0 cyclic (a single three-cycle), the skill ladder is 1.0/0.0. The **SVD rank-1** method the original plan suggested instead reports RPS as ≈ 0.707 transitive — wrong, because the singular values of an antisymmetric matrix come in equal pairs, so a rank-1 truncation captures at most half of its squared norm for *any* game (a pure skill ladder also scores 0.707); the Hodge method is the one used for the diagnosis. The two *real* populations diverge sharply: the **PSRO best-response meta-game on Leduc is mostly cyclic** (≈ 0.41 - 0.46 transitive, 27 three-cycles), while the **league’s snapshot meta-game is mostly transitive** (≈ 0.94 - 0.98).

Conclusion. The transitive/cyclic ratio is a *pre-training diagnostic*. Two populations drawn from the same game (Leduc) have opposite structure depending on how they are built (best responses vs training snapshots — see §8.2). This is exactly the tool that predicts whether naive self-play/PBT will converge or cycle, and it is the diagnostic the thesis carries into later chapters.

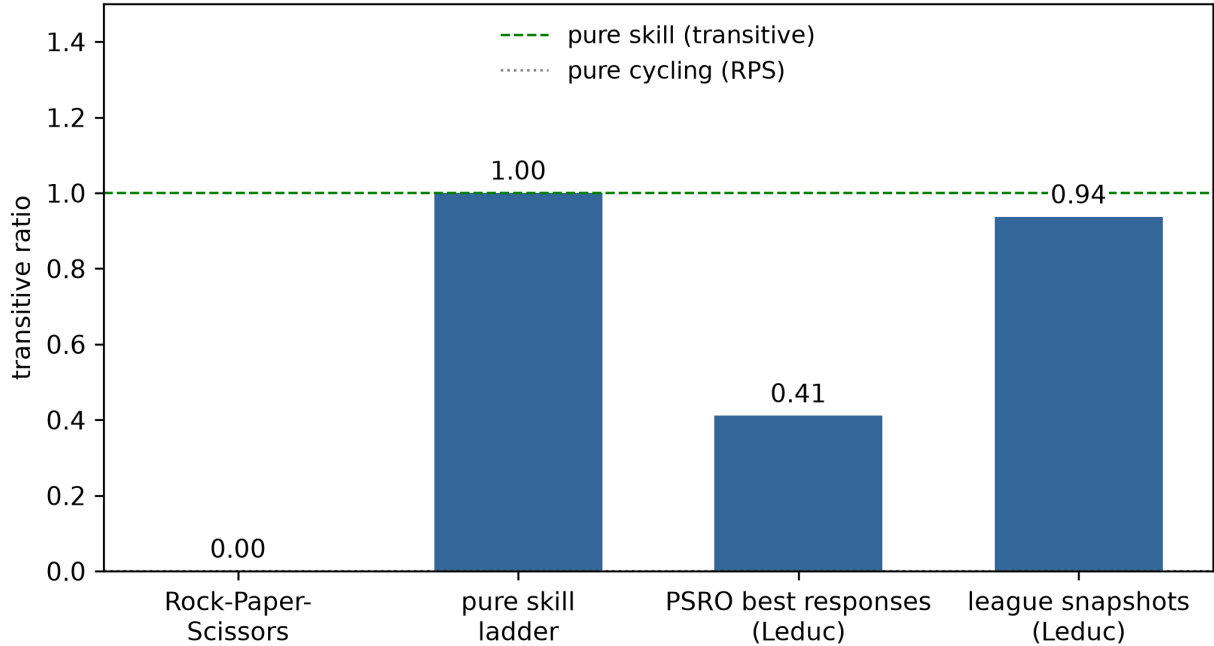


Figure 2: Transitive-ratio bar chart: Rock-Paper-Scissors (0.0, purely cyclic), a pure skill ladder (1.0, purely transitive), the PSRO-Leduc best-response meta-game (~ 0.41 - 0.46 , mostly cyclic with 27 three-cycles), and the league snapshot meta-game (~ 0.94 - 0.98 , mostly transitive). Which population you decompose decides whether Leduc looks like a wheel or a ladder.

3 PART II — THE PBT LEAGUE

3.1 4. What the league is and how it is graded

The second half builds a small **AlphaStar-style league** on Leduc Hold'em with three agent types — **main** agents (the product), **main exploiters** (hunt weaknesses in the current mains), and **league exploiters** (hunt weaknesses anywhere in the frozen history) — plus periodic **freezing** of snapshots into a museum and **PFSP** matchmaking (sample harder opponents more often). Agents are neural PPO networks on a Leduc info-state encoding; PBT copies the top agents (exploit) and perturbs their learning rate / entropy (explore). Crucially, each network is extracted to a **tabular** policy so Chapter 07's **exact** best response measures its exploitability — the same NashConv yardstick used since Chapter 2. NashConv is the sum of the two players' best-response gains; in OpenSpiel's convention exploitability is half of it, so every exploitability value in this report is a NashConv value and must be halved before it is compared with OpenSpiel-convention numbers. Two configs: **smoke** (7 live agents, 15 epochs) and **scale** (8 live agents, 120 epochs, 48 frozen snapshots).

3.2 5. Experiment 3 — league training dynamics

What we test. Does the league’s exploitability fall as training proceeds, and does the meta-Nash of the population track that improvement? (Raw step validation, L535, L537.)

How. Every epoch, record the minimum exploitability across the main agents and the exploitability of the current meta-Nash mixture; also record the league snapshot meta-game’s transitive ratio and final Elo. Data: `results/{smoke,scale}_results.json` (league block).

Metric	Smoke (15 epochs)	Scale (120 epochs)
min-main exploitability	4.67 $\rightarrow$ 3.04 (monotone, ends at min)	4.73 $\rightarrow$ min $\approx$ 1.21 (ep 66) $\rightarrow$ 2.05 (ep 119)
meta-Nash exploitability	4.73 $\rightarrow$ 3.04	5.01 $\rightarrow$ min $\approx$ 1.32 (plateau 1.60) $\rightarrow$ 2.96 (ep 119)
league meta-game transitive ratio	0.981	0.937
final Elo (live agents)	1176-1211	1198-1210

Results. Smoke shows a clean monotone decline and ends at its minimum (3.04) — but 15 epochs is too short to show what happens next. Scale reveals it: exploitability falls steeply within $\sim$ 15 epochs and is lowest around epochs 20–40 (meta-Nash $\approx$ 1.32 at epoch 21; best snapshot from epoch 29; the single lowest **min-main** value, $\approx$ 1.21, at epoch 66, is an isolated dip), then holds a $\approx$ 1.60 meta-Nash plateau, and then **regresses** back up to $\approx$ 2.05 (min-main) and $\approx$ 2.96 (meta-Nash) by epoch 119. Elo spread is compressed (all live agents within $\approx$ 15 points), consistent with a behaviorally-similar population.

Conclusion. The league *does* drive strong early improvement — but improvement is **not monotone at scale**: late in training the live main agents lose ground on absolute exploitability. Exploiter pressure may contribute, but the self-play baseline, which has no exploiters, also regresses after epoch 100, so it is not the only cause. The best agents are captured as *frozen snapshots* from the first half of the run. This non-monotonicity is the honest headline of the scale run and is reconciled in §8.3.

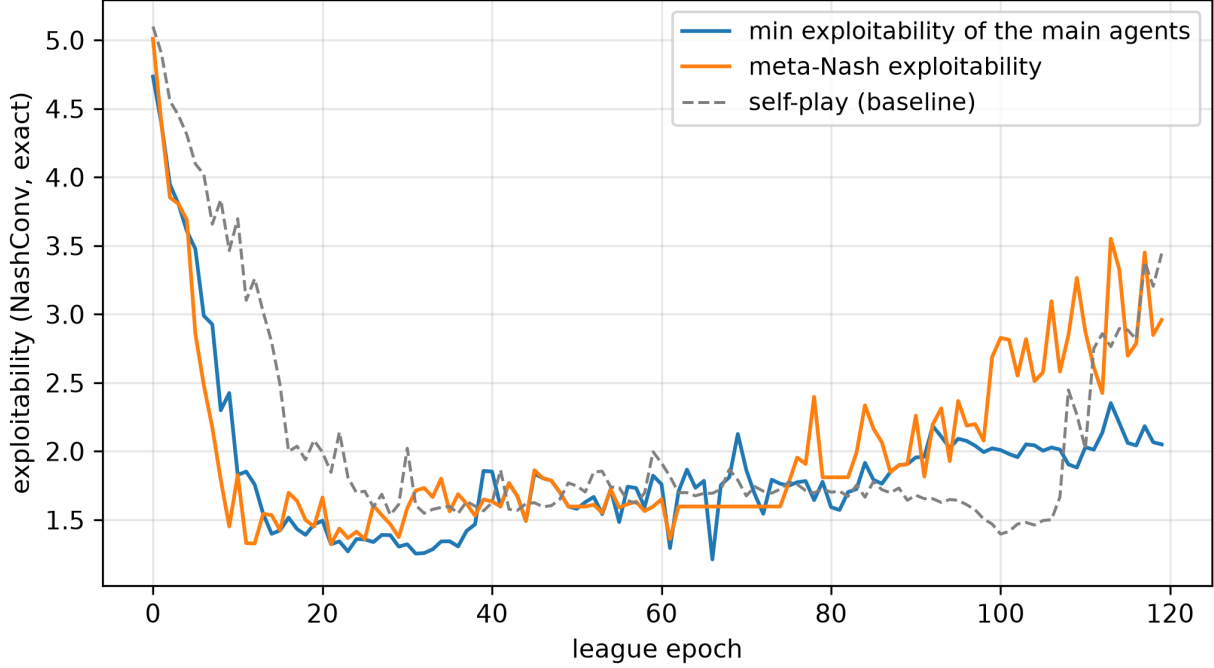


Figure 3: League exploitability (NashConv) over training (scale, 120 epochs): min-main and meta-Nash exploitability both fall steeply within ~ 15 epochs, are lowest around epochs 20-40 (meta-Nash ~ 1.32 at epoch 21), and then regress upward (~ 2.05 / ~ 2.96) by epoch 119. The dashed grey line is the self-play baseline, which also regresses after epoch 100. A running league is not a monotonically improving one; the best agents are frozen snapshots from the first half of the run.

3.3 6. Experiment 4 — EGTA, meta-Nash, and diversity

What we test. (a) Is the meta-Nash of the final population less exploitable than any individual agent (the raw-step expectation, L536)? (b) How diverse is the population (effective size, behavioral clustering, exploit coverage — L538)?

How. Build the empirical symmetric payoff matrix over all agents (live + frozen), solve the meta-Nash mixture, collapse it to a single behavioral policy, and measure its exact exploitability against the best individual’s. Diversity: effective population (participation ratio of meta-Nash weights), single-linkage behavioral clustering, and exploit coverage. Data: `results/{smoke,scale}_results.json` (league.egta, league.diversity). Toy intuition: `exploration/figures/mini_pbt.json`.

(a) *Meta-Nash vs best individual:*

Config	meta-Nash exploitability	best individual	meta-Nash $\leq$ best?
Smoke	2.665	2.665	yes (all weight on the best agent)
Scale	3.418	1.305	no (weight spread 0.645+tail)

(b) *Diversity:*

Metric	Smoke	Scale
agents / active (participation)	16 / 1 (1.00)	56 / 3 (1.92)
behavioral clusters (thr 0.30)	1 (max pairwise 0.255)	1 (max pairwise 0.484)
exploit coverage	0.625	0.571

Results. (a) In smoke the meta-Nash puts all weight on the single best agent, so meta-Nash = best = 2.665 (the prediction holds trivially). At scale the meta-Nash spreads weight and its collapsed mixture is *more* exploitable (3.418) than the population’s best single member (1.305), which gets zero weight — the prediction **fails** (§8.1). (b) The population is only weakly diverse: participation ratio rises from 1 to 1.9, but behavioral clustering collapses everything into a **single cluster** at both scales (even though the scale population’s max pairwise distance 0.484 exceeds the 0.30 threshold — single-linkage merges the chain). The mini-PBT toy makes the mechanism vivid: on transitive Prisoner’s Dilemma diversity collapses to 0; on cyclic RPS it churns forever (0.07-0.29).

Conclusion. EGTA gives a population-level exploitability, but two lessons emerge: the meta-Nash mixture is **not** guaranteed to be less exploitable than the population’s best member (§8.1), and PBT diversity here is *weight-level*, not *behavior-level* — the agents are near-identical, so the “population” is thinner than its size suggests.

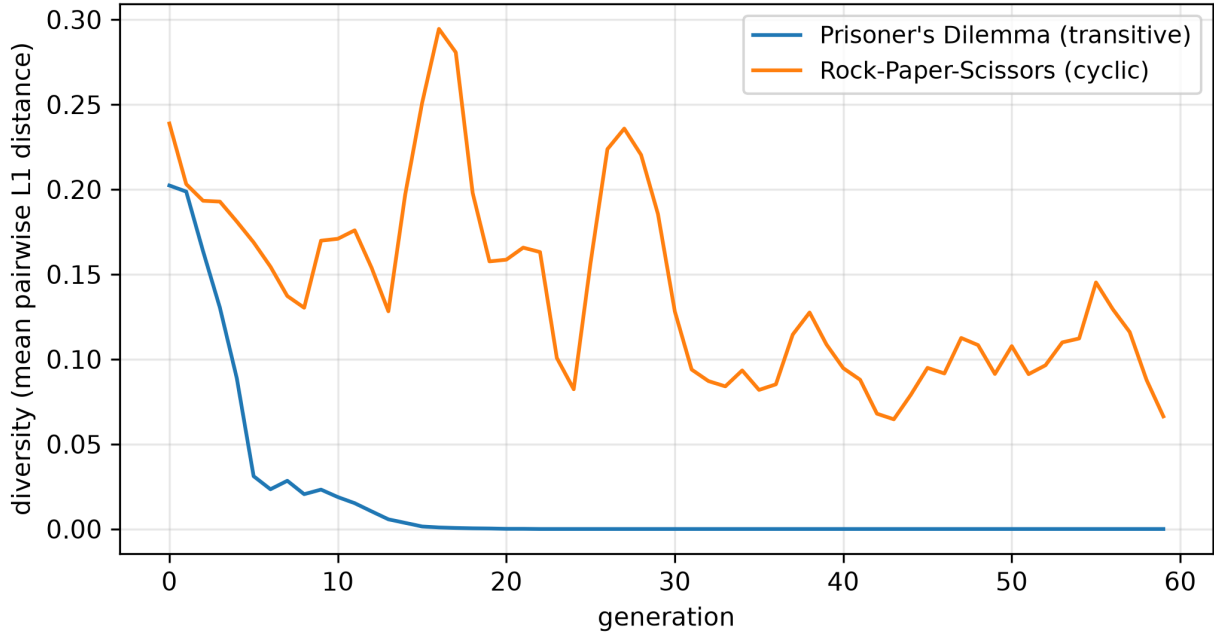


Figure 4: Mini-PBT diversity over generations: on the transitive Prisoner’s Dilemma the population’s strategy diversity collapses to zero (everyone converges to Defect), while on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29) as the population chases the wheel of counters. Game structure sets whether diversity survives (only one population size was run).

3.4 7. Experiment 5 — league vs PSRO vs self-play vs CFR-Nash

What we test. How does the league compare to the alternatives — exact PSRO, plain self-play, and CFR-Nash — on Leduc exploitability? (Raw step validation, L539.)

How. Run each method to its config’s budget and read the final exploitability. The league entry is its **best individual** (the strongest frozen snapshot, i.e. best of the run); self-play is reported both at its final iterate and at its best per-epoch iterate, so that best-of-run is compared with best-of-run; PSRO uses the exact BR oracle (its final value is also its best); CFR-Nash is the approximate-Nash floor. Data: `results/{smoke,scale}_results.json` (league, baselines).

Method	Smoke	Scale	Note
CFR-Nash (floor)	0.033 (2k iters)	0.0099 (20k iters)	approximate Nash, the target floor
League — best individual	2.665	1.305	strongest frozen snapshot (best of run)

Method	Smoke	Scale	Note
Self-play — best iterate	3.111	1.396 (ep 100)	best of run, like-for-like with the league entry
PSRO (exact BR)	3.037 (12 rounds)	2.163 (20 rounds)	Chapter 09’s slow-Leduc wall
League — meta-Nash mixture	2.665	3.418	see §8.1
Self-play — final agent	3.140	3.683	plain best-response- to-latest

Results. At scale the league’s **best individual** (1.305) is the lowest of the learned methods — better than exact PSRO (2.163) and the final self-play agent (3.683), but comparable to self-play’s own best iterate (1.396 at epoch 100; one run each, so the gap is not meaningful) — and all remain far above the CFR-Nash floor (0.0099). The league’s **meta-Nash mixture** (3.418) is worse than PSRO and the best individual, though better than the final self-play agent (§8.1). Self-play’s final agent actually *worsens* from smoke to scale, consistent with last-iterate non-convergence: at scale it reaches 1.396 and then degrades over the last 20 epochs.

Conclusion. The league is a good producer of strong *individuals* — its best snapshot beats PSRO and the final self-play agent and matches self-play’s best iterate — but the *mixture* is not, and none of the learned methods approach the exact-CFR floor on Leduc. The right thing to ship from a league is a *selected member*, not the meta-Nash mixture (§8.1).

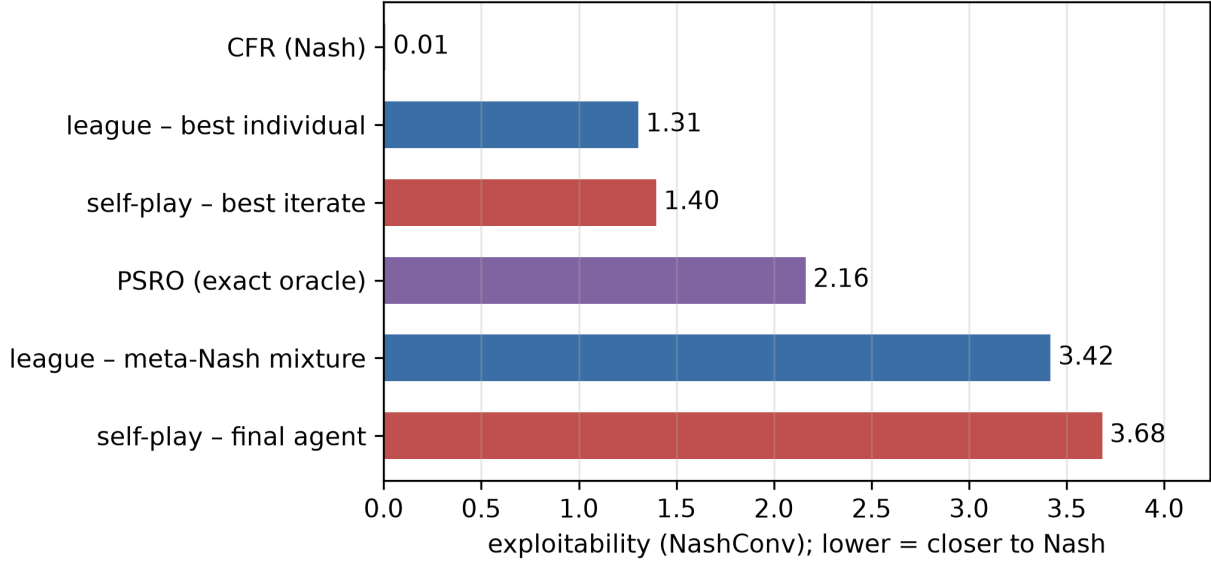


Figure 5: Leduc exploitability (NashConv) by method (scale): CFR-Nash floor ~ 0.01 ; the league’s best individual (1.31, best of the whole run) is comparable to self-play’s best iterate (1.40) and below PSRO (2.16), the league’s meta-Nash mixture (3.42) and the final self-play agent (3.68). Ship a selected member, not the mixture.

3.5 8. Prediction $\leftrightarrow$ reality reconciliation

Per WORKFLOW §0.1, contradicted Phase-4 predictions are kept and reconciled, not silently edited. Three gaps.

1. **Meta-Nash of the league $\leq$ best individual (§6a).** *Predicted* (raw step Exit Checklist, L536): the meta-Nash is less exploitable than any individual agent. *Measured*: true in smoke ($2.665 = 2.665$, all weight on the best agent), **false at scale** ($3.418 > 1.305$). *Reconciliation*: not a mixing bug — the identical `mixture_behavioral_policy` code path gave meta = best in smoke. The meta-Nash minimizes **meta-game regret** (win against the population), which is a *different objective* from minimizing **full-game exploitability**. The mixture is in fact *less* exploitable than each of its three components (3.558, 3.932, 4.262, with weights 0.322, 0.645, 0.032); the least exploitable agent (1.305, `main_2#e29`) and the other early snapshots simply received zero meta-Nash weight. The lesson inverts and sharpens: population evaluation must measure the collapsed mixture’s exploitability directly and, in practice, ship a selected member rather than the mixture.
2. **Leduc’s meta-game is transitive (§3).** *Predicted*: the intuition doc framed poker as a skill ladder. *Measured*: the PSRO **best-response** meta-game is mostly cyclic (≈ 0.41 -

0.46 transitive, 27 three-cycles), while the league **snapshot** meta-game is mostly transitive (≈ 0.94 - 0.98). *Reconciliation*: not a bug — a population of best responses cycles (A beats the mixture, B beats A, a later BR beats B; Czarnecki et al.’s spinning top), whereas a population of training-trajectory snapshots forms a ladder because later snapshots are mostly stronger. The structure is a property of the *population*, not just the game; both measurements are correct and instructive.

3. **League exploitability decreases monotonically (§5).** *Predicted*: a monotone decline. *Measured*: smoke (15 epochs) declines cleanly and ends at its minimum (3.04); scale (120 epochs) falls steeply within ~ 15 epochs, is lowest around epochs 20–40 (meta-Nash ≈ 1.32 at epoch 21, best snapshot from epoch 29; single min-main low ≈ 1.21 at epoch 66; meta-Nash plateau 1.60) and then **regresses** to ≈ 2.05 / ≈ 2.96 by epoch 119. *Reconciliation*: an honest, likely-real training instability, only visible once training is long enough — the live main agents lose absolute-exploitability ground late (churn / partial forgetting); exploiter pressure may contribute, but self-play without exploiters also regresses after epoch 100, so it is not the only cause. The best agents are the frozen snapshots from the first half of the run, which is exactly why the best-individual metric (§7) is strong while the late live agents are not. Documented, not fixed; the natural remedy is best-snapshot retention / population regularization (§11). This echoes Chapter 09’s methodological rule: **scale reveals what smoke hides**.

3.6 9. Trustworthiness and sample adequacy

- **Evolutionary results are bounded by analytic references.** Each replicator outcome is checked against the game’s analytic ESS/Nash; the spinning-top ratios are checked against the pure cases (RPS = 0.0, skill ladder = 1.0). These are deterministic and exactly reproducible.
- **League exploitability is Chapter 07’s *exact* NashConv**, not a simulation: every neural policy is extracted to a tabular policy and scored by exact best response, so “how exploitable is this agent/mixture?” is ground-truthed rather than estimated.
- **Neural training is seed- and library-sensitive.** The league is one PBT run per config; the *qualitative* findings (early improvement; best snapshot < PSRO < final self-play agent; late regression at scale; meta-Nash mixture > the population’s best member at scale) are the trustworthy claims, not the third-decimal magnitudes.
- **Diversity metrics are threshold-sensitive.** The single-cluster result depends on the 0.30 single-linkage threshold; it should be read as “behaviorally similar,” not “provably one strategy.”

3.7 10. Limitations (ranked by how much they affect the conclusions)

1. **Late-training regression at scale (§5, §8.3)** — the league does not monotonically improve; until best-snapshot retention/regularization is added and re-run, the “league improves” claim holds only for the *first half* of training and for the *frozen best*, not the live agents.
 2. **Meta-Nash mixture more exploitable than the population’s best member, which it gives zero weight (§6a, §8.1)** — means population evaluation must not report the mixture as the league’s strength; the selected member is the right artifact.
 3. **Thin behavioral diversity (§6b)** — participation ratio 1.9 and a single behavioral cluster mean the “population” is near-degenerate; the diversity benefits reported for AlphaStar, whose league created almost 900 distinct players, do not appear at this scale.
 4. **Single-run neural results (§9)** — direction-robust, magnitude-uncertain.
 5. **Toy scale throughout** — Leduc and small matrix games; nothing here should be extrapolated to deep-RL leagues at AlphaStar scale.
-

3.8 11. Conclusions and research directions

Conclusions. Part I delivers the diagnostic: replicator dynamics reproduce every analytic ESS/Nash outcome (including RPS’s non-converging orbit), and the Hodge spinning-top decomposition cleanly separates skill from cycles — revealing that the *same* game (Leduc) is cyclic as a best-response population but transitive as a snapshot population. Part II builds the league and grades it with exact exploitability: it produces strong *individuals* (best snapshot 1.305, beating PSRO 2.163 and matching self-play’s best iterate 1.396; its final agent is 3.683 — one run each), but three predictions broke honestly — the meta-Nash mixture is *more* exploitable than the population’s best member at scale, the Leduc best-response meta-game is *cyclic* not transitive, and league exploitability *regresses* late in long training. The population machinery works, but carries no monotonicity or safety guarantee — which is precisely the thesis gap.

Research directions (each tied to a measured effect):

- *Best-snapshot retention / population regularization* — directly targets the late regression (§8.3); re-run to see whether the guarantee gap is a training or a design issue.
- *Report the selected member, or a diversity-regularized meta-solver* — the meta-Nash-mixing failure (§8.1) motivates either shipping a best-response-robust member or changing the meta-solver objective.
- *Carry the transitive/cyclic diagnostic to Chapter 11’s FFA games* — test the “large cyclic component” prediction directly, where naive PBT is expected to cycle.
- *Formalize population safety without a minimax anchor (Contribution #2)* — the exploiter mechanism is heuristic; §8.1/§8.3 show it does not guarantee a non-exploitable population.

3.9 12. Reproduction

From `implementation/step10/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL/SKIP against the raw-step targets)  
python validate.py
```

```
# full comparison runs -> results/{smoke,scale}_results.json  
python tournament.py --config smoke  
python tournament.py --config scale
```

```
# plots from a results JSON -> plots/*.png (needs matplotlib)  
python plotting.py --config scale
```

From `implementation/step10/exploration/` (Part I figures + JSON):

```
python replicator_playground.py      # replicator phase portraits + JSON  
python game_landscape.py             # transitive/cyclic ratios (RPS / skill / PSRO-Leduc)  
python mini_pbt.py                  # diversity collapse vs churn  
python psro_population_peek.py       # PSRO population dynamics on Leduc
```

Seeds are fixed in `config.py` (implementation) and each exploration script's config. The evolutionary and spinning-top results are exactly reproducible; the neural league results are direction-stable across seeds. All numbers in this report were read from `results/{smoke,scale}_results.json` and `exploration/figures/*.json`.